

日常：改動在個人 workspace 累積，depot 隔很久才收到一大包

這份文件回答：把版控當備份的團隊，每天具體怎麼做事；問題從這些做法的哪裡長出來 ↓

個人 workspace

p4 client，只有自己看得到



submit 一大包

Perforce depot

大家看得到的那份



這段期間 depot 沒有任何變化

什麼時候才 submit

里程碑到了

有人跟你要

休假前

怕 workspace 壞掉

主管提醒

一包裡面有什麼

好幾個 block 的 RTL

順手改的 script

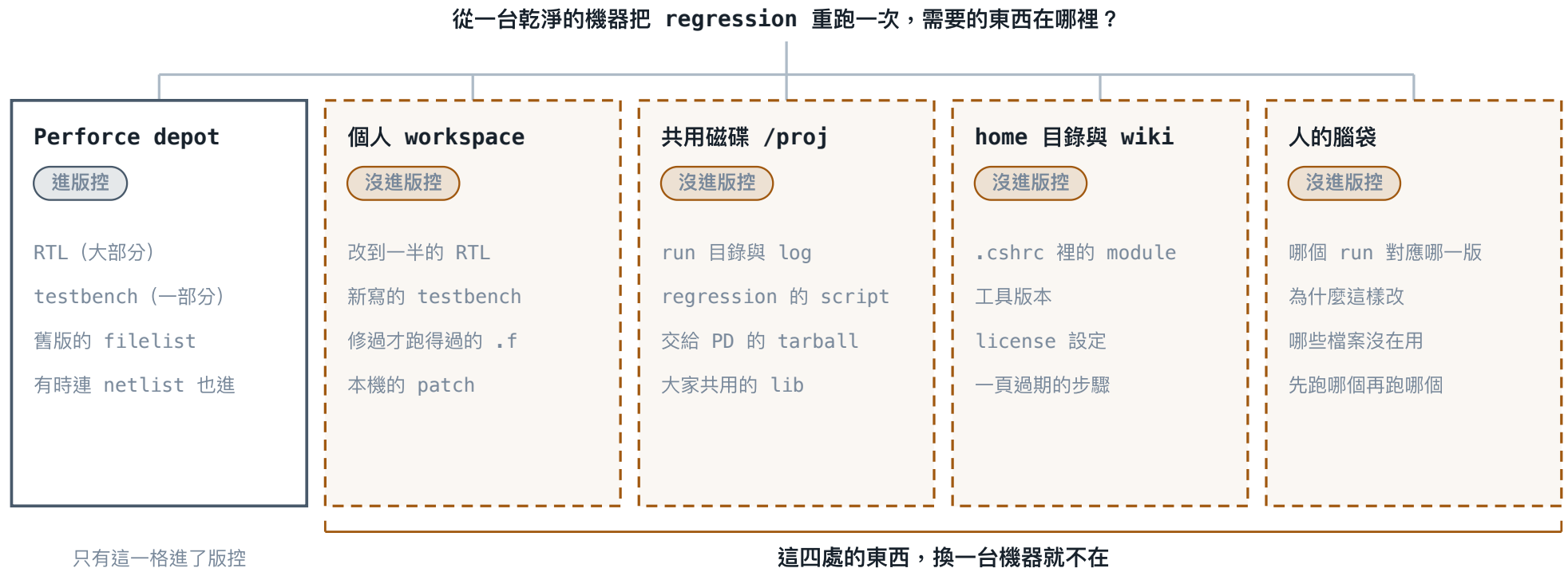
新的 testbench

一起改的 .f

submit 的目的是把檔案存起來，所以在怕丟、有人要、里程碑到了的時候才做。

兩次 submit 之間的所有改動，只存在這個人的 workspace 裡，別人看不到也用不到。

散落：跑得起來需要的東西分在五個地方，depot 只是其中之一



depot 裡只有檔案的一部分；要跑得起來，其餘的東西散在磁碟、home 目錄和人的腦袋裡。

換一台乾淨的機器，照 depot 的內容做不出同一個結果。

交付：結果靠 email 裡的路徑傳遞，label 只記得檔案

一次 regression 的結果怎麼傳出去



里程碑時打一個 label

label RTL_FREEZE_v2

它記得

- ✓ 哪些檔案、各是哪個版本

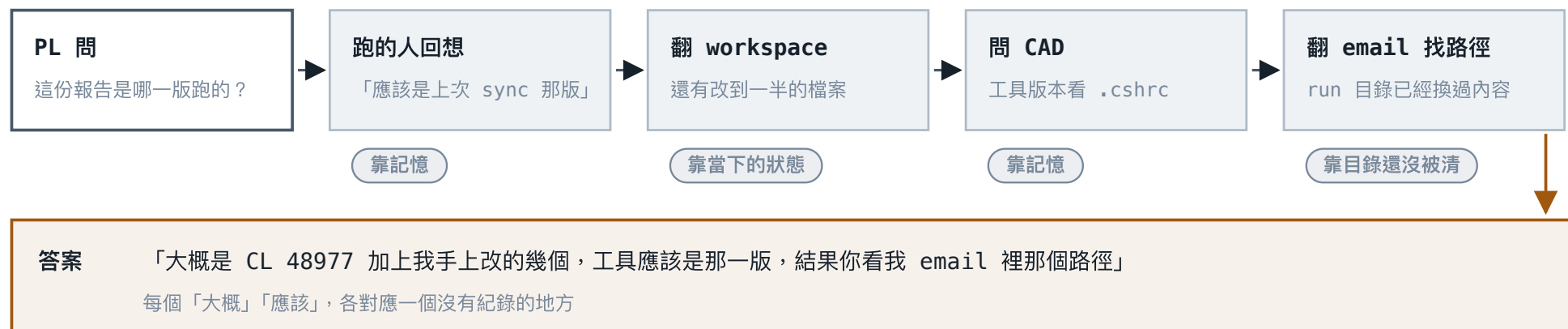
它不記得

- x simulator 與 synthesis 工具的版本
- x 環境變數與 license 設定
- x 跑了哪些測試、結果如何
- x 怎麼跑：script 在誰的目錄

結果和它的來源之間沒有連結：email 裡的路徑指向一個會變的目錄，label 只記得檔案。

想知道某個結果是怎麼來的，只能找到跑的那個人問。

問題：「這份結果是哪一版跑的」 要問好幾個人，答案仍是大概



沒有紀錄的地方

改動只在 workspace，沒有 CL 號碼

label 沒記工具版本

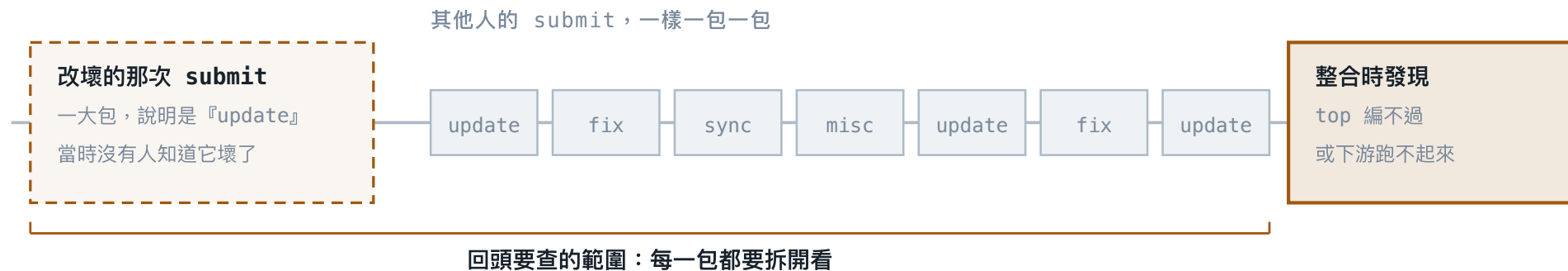
run 目錄會被覆蓋

跑法在個人的 script 裡

沒有任何一筆紀錄把結果、檔案版本、工具版本和跑法綁在一起。

答案由幾個人的記憶拼出來，每一段都帶著「應該」。

問題：壞掉被發現時，離改壞它的那次 submit 已經很遠



怎麼找

- 一包一包拆開，猜哪幾個檔案有關
- 逐個問當時改了什麼
- 在自己的 workspace 把舊版換回來試

誰先發現

整合的人 PD DV 韌體 客戶

發現的人離那次改動最遠，手上沒有線索

改動進 depot 時沒有任何檢查，問題要等到整合或下游才浮現。

從發現點往回找，中間每一包都要拆開看；一次送進的東西越多，越難切。

問題：下游說不出收到了什麼，人走了流程跟著走

交給下游



下游問不出來的

- ✗ 跟上一包差在哪？
- ✗ 哪些是刻意改的、哪些是順手改的？
- ✗ 這包和 label 的內容一樣嗎？
- ✗ 出問題了，要回報哪一版？

人離開的時候



下游收到的是一包檔案，拿不到它的來歷。

工作流程存在人身上，人離開的時候，depot 留不住它。

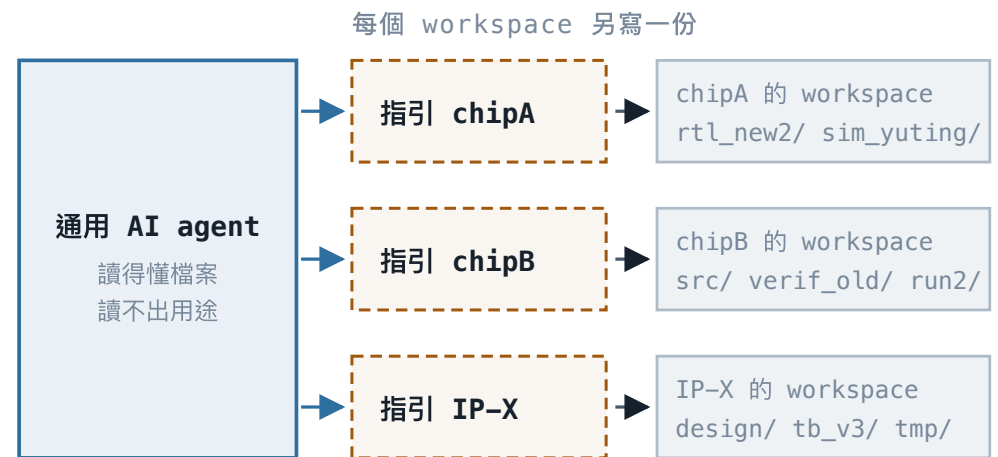
問題：目錄的用途靠人帶路，AI agent 每個 workspace 都要另寫指引

第一次打開專案目錄看到的：用途與相依都要猜

/proj/chipA/	
rtl/	? 哪些檔案還在用?
rtl_old/	? 能刪嗎?
rtl_new2/	? 跟 rtl/ 差在哪?
sim/	
sim_yuting/	? 個人的，還是正式的?
scripts/	? 哪個才是正式的?
scripts_bak/	
release_0917/	? 跟 label 的內容一樣嗎?
tmp/	
top.f	? 引用的檔案一半在 rtl_new2/
README	? 寫的是上一個專案

新來的工程師：問同事、慢慢摸；摸出來的地圖留在他腦袋裡

通用的 AI agent 進到這些 workspace



指引誰寫得出來？

只有知道的那幾個人

有幾個 workspace 就要寫幾份

目錄的用途與相依關係沒有寫下來，新人靠問、靠摸；地圖最後還是只在人的腦袋裡。

通用的 AI agent 一樣看不懂，每個 workspace 都得由知道的人另寫一份指引。

總結：備份做到了，關於檔案的問題一個都答不出

depot 做到的

- ✓ 檔案的最新版在哪
- ✓ 誰在什麼時候改過哪個檔案
- ✓ 舊版救得回來
- ✓ 里程碑那天的檔案清單 (label)

備份要做的事，它都做到了

depot 答不出來的

- x 這個結果是哪一版做的？
- x 從乾淨的機器能重現嗎？
- x 哪一次改動弄壞了它？
- x 下游收到的跟上一版差在哪？
- x 跑的步驟與環境在哪裡？
- x 這個檔案還有人在用嗎？
- x 這個目錄裝什麼、靠哪些東西？

今天誰在回答

- 跑的人
- 沒人試過
- 大家一起猜
- 上游工程師
- CAD 加跑的人
- 最資深的人
- 帶你的那個人

備份的功能它確實做到了；右邊每一題，今天都由某個人的記憶回答。

AI 的產出要跨團隊被採用，先得讓紀錄能回答右邊這些題。

收斂：七個問題歸到五個原則，之後的對策各自對應其中一個

原則	意思，以及做得到 / 做不到的檢驗	沒做到時的問題
小步常進	改動小而頻繁地進到共用的地方，每一包說得出改了哪一件事 檢驗：隨便挑一包 submit，說得出它改了哪一件事	圖 1 圖 5
單一事實來源 (SSOT)	跑得起來需要的一切都在版控裡，而且只有一份 檢驗：換一台乾淨的機器，只靠版控的內容做出同一個結果	圖 2 圖 3 圖 6
可追溯	每個結果與交付物都連得回產生它的版本、工具、環境與步驟 檢驗：隨便拿一份結果，說得出它的版本、工具、環境與步驟	圖 3 圖 4 圖 6
變更即驗證 (CI)	改動進來的當下就被機器檢查 檢驗：改壞的那一包進來時就被標出來，用不著等到整合	圖 5
自我描述	目錄的用途、相依、怎麼跑，寫在 repo 裡 檢驗：第一次來的人或 AI agent 只讀 repo，就說得出每個目錄的用途與相依	圖 6 圖 7

五個原則都是 repo 該有的性質，各有一個做得到或做不到的檢驗。
之後談對策，每一條只回答一個問題：它讓哪一個檢驗從做不到變成做得到。